

AKADEMIA LEONA KOŹMIŃSKIEGO · INFORMATYKA, I ROK · 03.11.2026

Lab 05

Struktury danych I: listy i krotki

dr inż. Karol Chlasta · WarsawIQ

Repozytorium kursu: github.com/KarolChlasta/KU-PyDev

Cel zajęć

- Rozumieć, **czym jest struktura danych** i jak lista wygląda „w środku”
- Tworzyć i modyfikować listy, korzystać z indeksów i wycinków
- Odróżniać typy **zmienne** (lista) od **niezmiennych** (krotka)
- Zbudować listę zakupów i system rezerwacji miejsc
- Efekty z sylabusa: **U1, U3, KS1**

CZĘŚĆ I

Teoria: sekwencje

Struktura danych

Definicja. Struktura danych to sposób **organizacji danych w pamięci** razem z operacjami, które można na nich wykonywać – i ich kosztem.

TYP	UPORZĄDKOWANY	ZMIENNY	DUPLIKATY	PRZYKŁAD
list	tak	tak	tak	[3, 1, 3]
tuple	tak	nie	tak	(2, 5)
str	tak	nie	tak	"ALK"
dict	tak (od 3.7)	tak	klucze – nie	lab 06
set	nie	tak	nie	lab 06

Lista „w środku”

- W CPythonie lista to **tablica dynamiczna wskaźników** do obiektów
- Elementy leżą w pamięci obok siebie → dostęp po indeksie jest natychmiastowy
- Przy `append` lista rezerwuje **zapas miejsca**, żeby nie kopiować się co chwilę

OPERACJA	KOSZT
<code>lst[i], lst[i] = x</code>	stały, $O(1)$
<code>lst.append(x)</code>	stały (średnio), $O(1)$
<code>lst.insert(0, x), lst.pop(0)</code>	liniowy, $O(n)$ – przesuwa resztę
<code>x in lst, lst.remove(x)</code>	liniowy, $O(n)$ – szuka po kolei

Python Wiki, „TimeComplexity”; kod źródłowy CPython: `Objects/listobject.c`

Indeksy i wycinki

```
letters = ["a", "b", "c", "d", "e"]
#           0     1     2     3     4
#          -5    -4    -3    -2    -1
```

WYRAŻENIE	WYNIK
<code>letters[0], letters[-1]</code>	<code>'a', 'e'</code>
<code>letters[1:3]</code>	<code>['b', 'c']</code> – stop nie wchodzi
<code>letters[:2], letters[-2:]</code>	<code>['a', 'b'], ['d', 'e']</code>
<code>letters[::-2]</code>	<code>['a', 'c', 'e']</code>
<code>letters[::-1]</code>	<code>['e', 'd', 'c', 'b', 'a']</code>

- Indeks poza zakresem → `IndexError`; **wycinek** poza zakresem → po prostu krótszy wynik

Najważniejsze metody list

METODA	DZIAŁANIE	ZWRACA
<code>append(x)</code>	dodaje na koniec	<code>None</code>
<code>extend(iter)</code>	dokleja wiele elementów	<code>None</code>
<code>insert(i, x)</code>	wstawia na pozycję <code>i</code>	<code>None</code>
<code>remove(x)</code>	usuwa pierwsze <code>x</code> (<code>ValueError</code> , gdy brak)	<code>None</code>
<code>pop(i=-1)</code>	usuwa i zwraca element	element
<code>sort()</code>	sortuje w miejscu	<code>None</code>
<code>index(x)</code> , <code>count(x)</code>	pozycja, liczba wystąpień	<code>int</code>

Pułapka: `lst = lst.sort()` – teraz `lst` to `None`! Metody zmieniające listę zwracają `None`. Nową posortowaną listę daje `sorted(lst)`.

Zmienność i aliasy

```
a = [1, 2, 3]
b = a          # druga NAZWA tej samej listy
b.append(4)
print(a)       # [1, 2, 3, 4] - zmieniła się też „a”!

c = a.copy()   # albo a[:] - NOWA lista
c.append(5)
print(a)       # [1, 2, 3, 4]
```

- Przypisanie nie kopiuje – tworzy **alias** (pamiętasz: zmienna to etykieta)
- `copy()` robi kopię **płytką**: listy w liście nadal są wspólne (`copy.deepcopy` – kopia głęboka)

Krotki

```
seat = (2, 5)           # krotka
row, number = seat      # rozpakowanie
single = (7,)           # krotka jednoelementowa – przecinek!
seat[0] = 3              # TypeError: 'tuple' object does not support item assignment
```

- **Niezmienna** → bezpieczna do przekazywania, może być **kluczem słownika** i elementem zbioru
- Naturalna dla **rekordów** o stałej strukturze: punkt `(x, y)`, miejsce `(rząd, miejsce)`, data `(2026, 11, 3)`
- Krotki porównują się **leksykograficznie** – element po elemencie:

```
sorted([(2, 5), (1, 8), (2, 1)])    # [(1, 8), (2, 1), (2, 5)]
```

Lista czy krotka?

PYTANIE	LISTA	KROTKA
Czy zawartość się zmienia?	tak	nie
Czy elementy są „tego samego rodzaju”?	zwykle tak	często różne pola rekordu
Klucz słownika / element zbioru?	nie	tak
Przykład	lista zakupów	współrzędne miejsca

- Zasada praktyczna: **kolekcja rzeczy** → **lista**, **jedna rzecz z kilkoma polami** → **krotka**

CZĘŚĆ II

Praktyka

Program sterowany poleceniami

```
shopping = []
while True:
    command, _, item = input("> ").strip().partition(" ")
    if command == "dodaj":
        shopping.append(item)
    elif command == "pokaż":
        print(", ".join(shopping) if shopping else "Lista jest pusta")
    elif command == "koniec":
        break
```

- `partition(" ")` dzieli tekst na: przed spacją, spację, resztę
- Taki schemat (pętla + `if/elif` po poleceniach) to szkielet każdego **interfejsu tekstowego**

Ciekawostki

- `list.sort()` w Pythonie to **Timsort** – algorytm, który Tim Peters napisał dla Pythona w 2002 r.; później przejęły go m.in. Java (sortowanie obiektów) i Android
- Tim Peters jest też autorem **Zen Pythona** (`import this`)
- Nazwa *tuple* pochodzi od zakończeń *quintuple*, *sextuple*... – czyli „n-tka”
- Zajrzyj, jak lista rośnie „skokami”:

```
import sys
lst = []
for i in range(10):
    lst.append(i)
    print(len(lst), sys.getsizeof(lst))
```

Dzisiejsze zadania

PLIK	ZADANIE
★ <code>zad1_zakupy.py</code>	lista zakupów z poleceniami
★ <code>zad2_rezerwacje.py</code>	rezerwacja miejsc – krotki
★ <code>zad3_wycinki.py</code>	wycinki i funkcje na listach
☆ <code>zad4_znajdz_pare.py</code>	gra „znajdź parę”

Sprawdzenie i oddanie

```
cd lab05_listy_krotki
python -m unittest discover -s tests -v
git add .
git commit -m "Lab 05"
git push
```

- Punkt aktywności: **obecność + wszystkie testy ★ zielone + push na GitHub przed 11:45**
- Po zajęciach – **0 pkt**, bez wyjątków
- Pamiętaj: **Sync fork** na początku zajęć, żeby pobrać nowy lab

Do poczytania

- E. Matthes, *Python. Instrukcje dla programisty*, Helion 2023 – rozdz. 3–4
- M. Dawson, *Python dla każdego*, Helion 2015 – rozdz. 5
- L. Ramalho, *Zaawansowany Python*, Promise 2020 – rozdz. 2 (sekwencje)
- Python Wiki: *TimeComplexity*(wiki.python.org/moin/TimeComplexity)

Następny lab (24.11): słowniki i zbiory